

Análisis y predicción del nivel de significancia de terremotos mediante Machine Learning

-
- Alex Santiago Salamanca Toloza - 2231873
 - Camilo Andrés Velandia Mendoza - 2232728
 - Jhoan Sebastian Pinzón Mogotocoro - 2191958

22971 - INTELIGENCIA ARTIFICIAL I

Universidad
Industrial de
Santander



Introducción

1. El análisis sísmico permite estudiar el impacto de los terremotos a partir de variables como *magnitud*, *profundidad*, *intensidad percibida* y *nivel de alerta*.
2. Se trabajó con un dataset de **1000** eventos sísmicos registrados entre **1995** y **2023**.
3. La variable objetivo fue **sig**, que representa la significancia global del evento sísmico.
4. Se aplicaron modelos supervisados como *Decision Tree*, *Random Forest*, *SVM* y *Redes Neuronales MLP* para predecir **sig**.
5. También se usaron técnicas no supervisadas como **K-Means** y **DBSCAN**, tomando **alert** como referencia para la actividad.



Objetivos

Objetivo general:

Desarrollar y comparar modelos de Machine Learning para predecir la significancia (**sig**) de eventos sísmicos registrados entre **1995** y **2023**.

Objetivos específicos:

- Limpiar y preparar el dataset para el entrenamiento de los modelos.
- Implementar modelos supervisados: Decision Tree, Random Forest, SVM y MLP.
- Evaluar el desempeño usando RMSE, MAE y MSE.
- Aplicar clustering con K-Means, DBSCAN y PCA para analizar agrupamientos.
- Identificar los modelos y variables más relevantes en la predicción de sig.



01 Montamos el drive y cargamos el dataset

```
1 # @title
2 from google.colab import drive
3 drive.mount('/content/drive')
```

```
1 #@title Carga Dataset
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 %matplotlib inline
6 df = pd.read_csv('/content/drive/MyDrive/earthquake_1995-2023.csv')
7 df.head()
```

	title	magnitude	date_time	cdi	mmi	alert	tsunami	sig	net	nst	dmin	gap	magType	depth	latitude	longitude	location	continent	country
0	M 6.5 - 42 km W of Sola, Vanuatu	6.5	18-08-2023 12:47	7	4	green	0	657	us	114	7.177000	25.0	mww	192.955	-13.8814	167.1580	Sola, Vanuatu	NaN	Vanuatu
1	M 6.5 - 43 km S of Intipucá, El Salvador	6.5	19-07-2023 00:22	8	6	yellow	0	775	us	92	0.679000	40.0	mww	89.727	12.8140	-88.1285	Intipucá, El Salvador	NaN	NaN
2	M 6.6 - 25 km ESE of Loncopué, Argentina	6.6	17-07-2023 03:05	7	5	green	0	899	us	70	1.634000	28.0	mww	171.371	-38.1911	-70.3731	Loncopué, Argentina	South America	Argentina
3	M 7.2 - 98 km S of Sand Point, Alaska	7.2	16-07-2023 06:48	6	6	green	1	860	us	173	0.907000	36.0	mww	32.571	54.3844	-160.6990	Sand Point, Alaska	NaN	NaN
4	M 7.3 - Alaska Peninsula	7.3	16-07-2023 06:48	0	5	NaN	1	820	at	79	0.879451	172.8	Mi	21.000	54.4900	-160.7960	Alaska Peninsula	NaN	NaN

02 Resumen estadístico y columnas

`df.describe()` genera un resumen estadístico, se aplicó para analizar las variables numéricas de un dataset de sismos (como *magnitud*, *profundidad*, *latitud* y *longitud*).

- Muestra cantidad de datos, promedio, desviación estándar, mínimos, máximos y cuartiles (25%, 50%, 75%).
- Muestra cantidad de datos, valores únicos, el más frecuente (**top**) y su frecuencia.

El atributo **df.columns** se utiliza para listar los nombres de todas las columnas que componen el DataFrame.

Al ejecutarlo obtenemos una lista (un objeto Index) con los 19 campos del dataset de sismos

```
1 #@title Columnas
2 df.columns

Index(['title', 'magnitude', 'date_time', 'cdi', 'mmi', 'alert', 'tsunami',
      'sig', 'net', 'nst', 'dmin', 'gap', 'magType', 'depth', 'latitude',
      'longitude', 'location', 'continent', 'country'],
      dtype='object')
```

```
1 #@title Resumen estadístico
2 df.describe()
```

	magnitude	cdi	mmi	tsunami	sig	nst	dmin	gap	depth	latitude	longitude
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	6.940150	3.805000	6.02700	0.325000	847.915000	193.918000	1.125174	20.926290	74.612541	4.315554	51.488576
std	0.438148	3.328972	1.43399	0.468809	301.802632	239.045858	2.073164	24.415895	130.812590	26.633320	117.478302
min	6.500000	0.000000	1.00000	0.000000	650.000000	0.000000	0.000000	0.000000	2.700000	-61.848400	-179.968000
25%	6.800000	0.000000	5.00000	0.000000	891.000000	0.000000	0.000000	0.000000	16.000000	-13.518500	-71.694450
50%	6.800000	4.000000	6.00000	0.000000	744.000000	0.000000	0.000000	18.000000	29.000000	-2.443500	107.791000
75%	7.100000	7.000000	7.00000	1.000000	874.250000	403.000000	1.549250	27.000000	55.000000	25.167250	148.384750
max	9.100000	9.000000	10.00000	1.000000	2910.000000	934.000000	17.854000	239.000000	870.810000	71.631200	179.682000

```
1 #@title Información Básica
2 df.info()
```

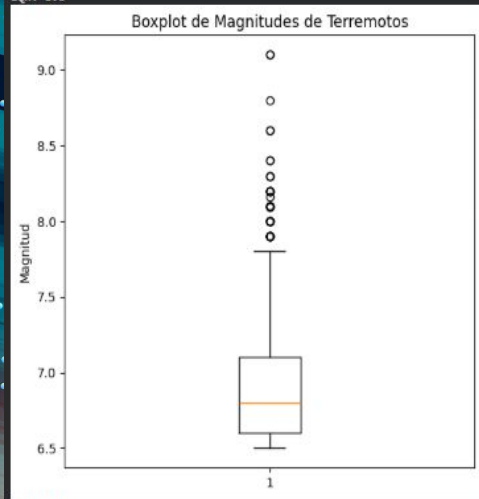
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 19 columns):
#   Column      Non-Null Count  Dtype
---  -
0   title       1000 non-null   object
1   magnitude   1000 non-null   float64
2   date_time   1000 non-null   object
3   cdi         1000 non-null   int64
4   mmi         1000 non-null   int64
5   alert       449 non-null    object
6   tsunami     1000 non-null   int64
7   sig         1000 non-null   int64
8   net         1000 non-null   object
9   nst         1000 non-null   int64
10  dmin        1000 non-null   float64
11  gap         1000 non-null   float64
12  magType     1000 non-null   object
13  depth       1000 non-null   float64
14  latitude    1000 non-null   float64
15  longitude   1000 non-null   float64
16  location    994 non-null    object
17  continent   284 non-null    object
18  country     651 non-null    object
dtypes: float64(6), int64(5), object(8)
memory usage: 148.6+ KB
```

3

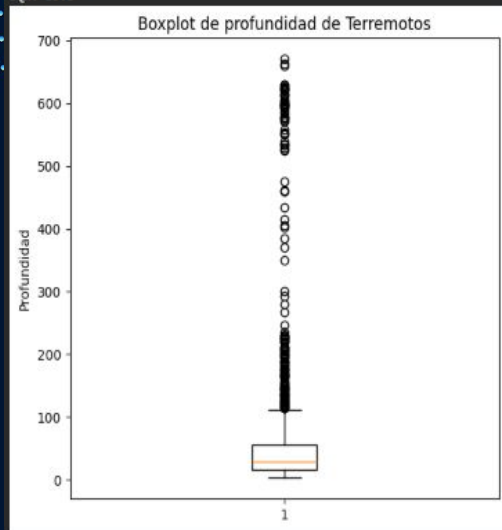
Información básica

- **df.info()** permitió revisar la estructura general del dataset.
- El conjunto cuenta con **1000** registros y **19** columnas.
- Se identificaron valores faltantes en variables como **alert**, **continent** y **country**.
- También se verificaron los tipos de datos: **numéricos** y **categoricos**.

Q1: 6.6
Q3: 7.1
IQR: 0.5



Q1: 16.0
Q3: 55.0
IQR: 39.0



Boxplot de magnitudes

- Se usaron diagramas de caja para analizar la distribución de magnitud y profundidad.
- El boxplot permite identificar la mediana, la dispersión de los datos y posibles valores atípicos.
- Se aplicó el método del rango intercuartílico (**IQR**) para detectar outliers.
- Los puntos fuera de los bigotes representan sismos con valores extremos.
- En profundidad se observa mayor cantidad de valores atípicos, lo que indica eventos sísmicos registrados a profundidades poco comunes.

Diagrama de pie

- Muestra la distribución de la variable **alert**.
- Permite observar qué niveles de riesgo son más frecuentes en el dataset.
- Ayuda a identificar si predominan sismos de baja, media o alta alerta.

Magnitud y profundidad de terremotos

- Se graficaron la magnitud y la profundidad para analizar patrones sísmicos.
- Estas variables permiten observar la intensidad del evento y qué tan profundo ocurrió.
- Los gráficos ayudan a detectar valores extremos o comportamientos relevantes en los datos.

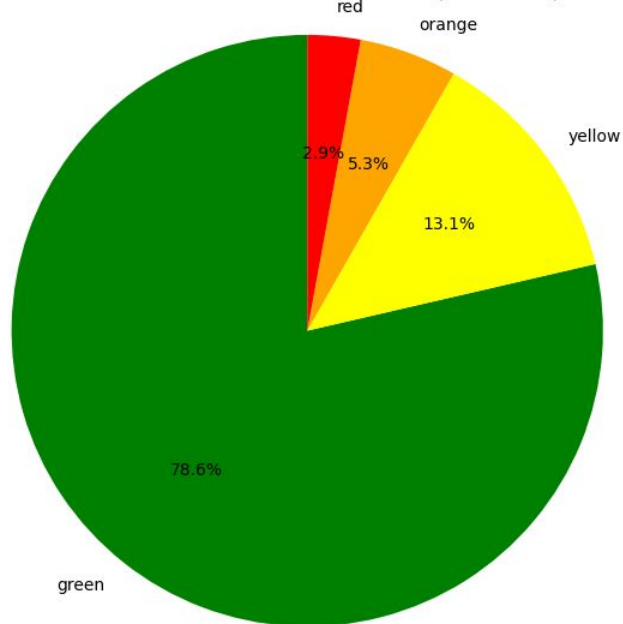
```
1 #@title Diagrama de pie para alertas
2 alert_counts = df["alert"].value_counts()
3
4 labels = alert_counts.index
5 sizes = alert_counts.values
6
7 colors = ["green", "yellow", "orange", "red"]
8
9 plt.figure(figsize=(7,7))
10 plt.pie(
11     sizes,
12     labels=labels,
13     autopct="%1.1f%%",
14     colors=colors,
15     startangle=90
16 )
17
18 plt.title("Distribución de Alertas de Terremotos (2001-2023)")
19 plt.axis("equal")
20
21 plt.show()
```

```
1 #@title Profundidad
2 n_primeros = 1000
3 indices = range(n_primeros)
4 magnitud = df['magnitude'].values[:n_primeros]
5 profundidad = df['depth'].values[:n_primeros]
6 plt.figure(figsize=(14,5))
7 plt.plot(indices, profundidad, color='green', alpha=0.7, linewidth=1.2)
8 plt.title('Profundidad de los Terremotos', fontweight='bold')
9 plt.xlabel('Índice del evento')
10 plt.ylabel('Profundidad (km)')
11 plt.grid(True, linestyle='--', alpha=0.5)
12 plt.tight_layout()
13 plt.show()
```

```
1 #@title Magnitud de terremotos
2 plt.figure(figsize=(10,5))
3 plt.plot(df["magnitude"])
4
5 plt.title("Magnitud de los Terremotos")
6 plt.xlabel("Número de evento")
7 plt.ylabel("Magnitud (Richter)")
8 plt.grid(True)
9
10 plt.show()
```

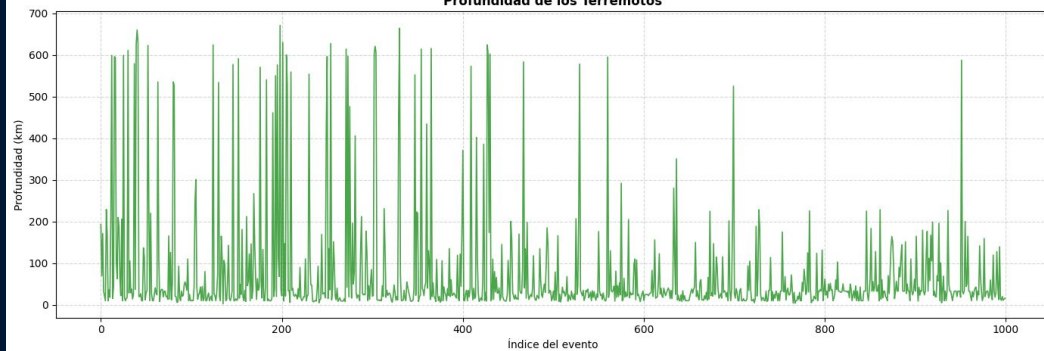

Diagrama de pie

Distribución de Alertas de Terremotos (2001-2023)

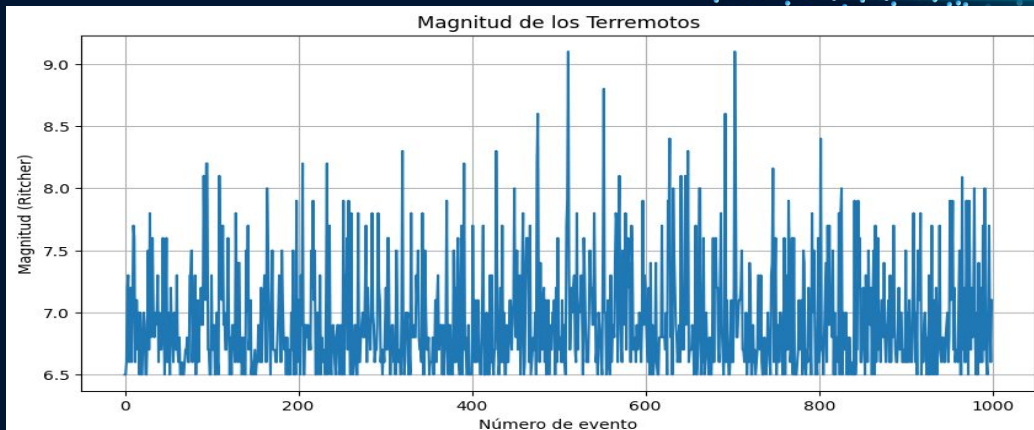


Magnitud y profundidad de terremotos

Profundidad de los Terremotos



Magnitud de los Terremotos

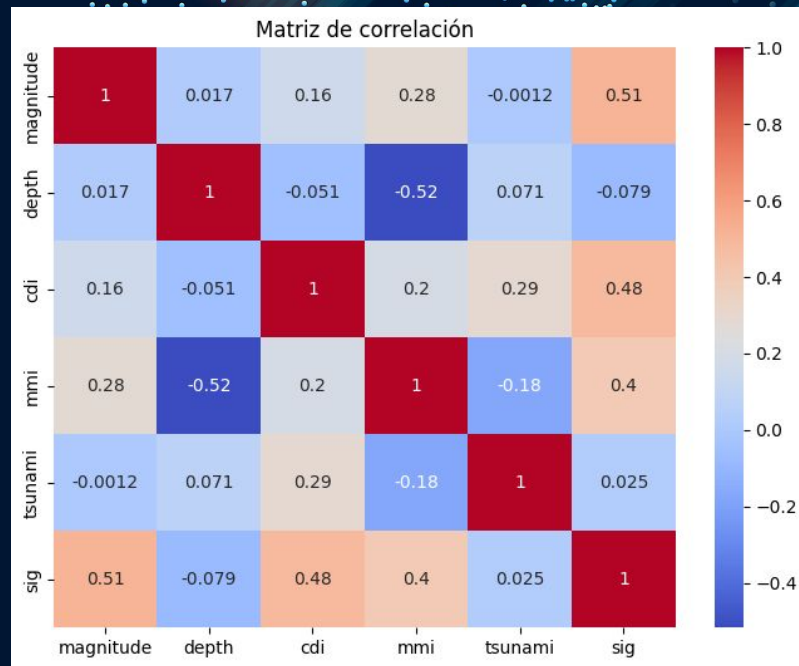


Matriz de correlación

- Se utilizó para medir la relación lineal entre las variables numéricas del dataset.
- Permite identificar qué variables están más asociadas con la significancia del sismo (**sig**).
- La correlación ayuda a entender qué características pueden aportar más información al modelo predictivo.
- En la matriz se observa que algunas variables, como **magnitud**, **cdi** y **mmi**, tienen mayor relación con **sig**.

¿Qué fórmula matemática utiliza?

- La función `.corr()` utiliza por defecto el **Coefficiente de Correlación de Pearson**.
- Sus valores van de -1 a 1:
 - Cercano a 1: relación positiva fuerte.
 - Cercano a -1: relación negativa fuerte.
 - Cercano a 0: poca relación lineal.



$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Procesamiento y limpieza de datos

- Se eliminaron registros duplicados para evitar que el modelo aprendiera datos repetidos.
- Se revisaron valores nulos con `isnull().sum()` y se limpiaron registros incompletos con `dropna()`.
- Con esto se obtuvo una base más consistente para entrenar los modelos predictivos.

Particionado 80/20

- El dataset se dividió en **80% entrenamiento** y **20% prueba**.
- **X_train / y_train**: datos usados para que el modelo aprenda los patrones.
- **X_test / y_test**: datos reservados para evaluar el desempeño del modelo.
- Esta división permite medir si el modelo puede predecir correctamente nuevos sismos.

```
Valores nulos antes de la limpieza (columnas relevantes):
magnitudo      0
depth          0
cdi            0
mmi            0
tsunami        0
sig            0
dtype: int64
```

```
Valores nulos después de la limpieza (columnas relevantes):
magnitudo      0
depth          0
cdi            0
mmi            0
tsunami        0
sig            0
dtype: int64
```

```
Filas duplicadas después de limpiar: 0
```

```
Dimensiones del dataset después de la limpieza: (938, 6)
```

```
Resumen de la variable objetivo 'sig':
count    938.000000
mean     857.646055
std      308.699974
min      650.000000
25%      691.000000
50%      754.000000
75%      889.000000
max      2910.000000
Name: sig, dtype: float64
```

```
Tamaño del conjunto de entrenamiento (X_train): (750, 5)
Tamaño del conjunto de prueba (X_test): (188, 5)
Tamaño del conjunto de entrenamiento y_train: (750,)
Tamaño del conjunto de prueba y_test: (188,)
```

MODELO 1: ÁRBOL DE DECISIÓN (Decision Tree)

- Se aplicó un *DecisionTreeRegressor* para predecir la significancia del sismo (**sig**).
- Se realizó tuning del hiperparámetro *max_depth*, buscando controlar la profundidad del árbol y evitar sobreajuste.
- Se compararon los resultados usando **RMSE**, **MAE** y **MSE**.
- La mejor configuración fue seleccionada según el menor error obtenido en las pruebas.

=== Resultados del tuning de Decision Tree ===

	max_depth	RMSE	MAE	MSE
2	3	246.365900	130.957758	60696.156478
3	4	250.076352	121.293627	62538.181740
4	5	252.472399	117.024027	63742.312286
1	2	254.933191	134.653559	64990.931710
6	8	260.883697	111.894255	68060.303112
5	6	261.110173	116.092822	68178.522379
0	1	269.842390	166.063268	72814.915230
7	10	281.278562	128.797452	79117.629466
8	None	289.982217	131.037234	84089.686170

=== Mejor modelo Decision Tree ===
Mejor max_depth: 3
RMSE 246.366
MAE 130.958
MSE 60696.156

Modelo 2: RandomForest

- Se buscó la mejor cantidad de árboles (*n_estimators*) para predecir **sig**.
- El modelo combina varios árboles de decisión y compara su desempeño usando RMSE y MAE.
- Según los resultados, **Random Forest** obtuvo uno de los mejores rendimientos, manteniendo buen equilibrio entre error y estabilidad.

Modelo 3: Máquinas de vectores de soporte SVM

- Se aplicó **StandardScaler** para normalizar las variables antes del entrenamiento.
- Se compararon los kernels *linear*, *polynomial* y *RBF* usando **RMSE**, **MAE** y **MSE**.
- El mejor resultado fue obtenido con kernel **RBF**, por lo que se seleccionó como configuración final del modelo **SVM**.

```
=== Resultados del tuning de Random Forest ===
```

	n_estimators	RMSE	MAE	MSE
2	25	232.207778	104.692908	53920.451956
6	150	234.056616	105.584141	54782.499379
5	100	234.977118	105.849570	55214.245847
7	200	235.132144	105.210947	55287.125299
8	300	235.146208	104.851246	55293.738942
3	50	236.360956	106.833514	55866.501638
4	75	237.162213	107.125112	56245.915272
1	10	239.631645	109.940957	57423.325053
0	5	275.096617	127.833511	75678.148564

```
=== Mejor modelo Random Forest ===  
Mejor n_estimators: 25  
MSE 53920.452  
RMSE 232.208  
MAE 104.693
```

```
=== Resultados SVM ===
```

	Kernel	RMSE	MAE	MSE
2	RBF	247.103993	104.491410	61060.383159
1	Polynomial	264.253377	125.021302	69829.847381
0	Linear	286.595559	123.405550	82137.014221

```
=== Mejor modelo SVM ===  
Mejor kernel: RBF  
RMSE 247.104  
MAE 104.491  
MSE 61060.383
```

Cross validation

Decision Tree

- Se evaluó con **K-Fold** $k = 10$ para probar el modelo en distintas particiones del dataset.
- Sirvió como modelo base para comparar el error frente a métodos más robustos.
- El **RMSE** promedio muestra qué tanto se desvía al predecir la variable **sig**.

```
=== Decision Tree | k = 10 ===
```

```
RMSE por fold: [245.17508749 251.56463549 305.25395102 216.87955389 205.85031993  
290.32562443 194.09894711 197.56206826 179.59786061 160.5633821 ]
```

```
RMSE promedio: 224.687
```

```
Desviación estándar: 44.915
```

SVM (RBF)

- Se usó kernel **RBF** para capturar relaciones no lineales.
- Obtuvo un RMSE promedio de **293.351**.
- Tuvo mayor error frente a **Decision Tree** y **Random Forest**.

```
=== SVM (RBF) | k = 10 ===
```

```
RMSE por fold: [323.59362943 338.49265289 431.1401991 276.35593497 229.61512502  
386.65614354 247.40193356 234.44566512 238.64436978 227.16497157]
```

```
RMSE promedio: 293.351
```

```
MAE promedio: 133.421
```

Random Forest

- Se evaluó con **10** folds, manteniendo varios árboles en cada entrenamiento.
- Obtuvo un **RMSE** promedio de **225.575**.
- Mantuvo buena estabilidad en los diferentes escenarios.

```
=== Random Forest | k = 10 ===
```

```
RMSE por fold: [237.60530833 236.26009326 300.85924377 199.7711533 210.32052843  
261.43034593 230.5350576 178.47340848 205.79350059 194.69725219]
```

```
RMSE promedio: 225.575
```

```
Desviación estándar: 34.203
```

MODELO 4: Deep Learning Perceptrón multicapa

Modelo 4.1: MLP con capas ocultas (64, 128, 128)

- Primera red neuronal entrenada para predecir **sig** usando las variables **magnitude**, **depth**, **cdi**, **mmi** y **tsunami**.
- Se usó una arquitectura secuencial con capas **Dense** y activación **ReLU**, permitiendo capturar relaciones no lineales entre las variables.
- El optimizador **Adam** ayudó a ajustar el aprendizaje de forma automática, teniendo en cuenta que el dataset es pequeño y se entrenó con pocas épocas.

Modelo 4.2: MLP con capas ocultas (64, 64, 128, 128, 256, 256)

- Se probó una red más profunda para capturar mejor relaciones entre **magnitud**, **CDI**, **tsunami** y valores altos de **sig**.
- El modelo redujo el **MAE**, pero aumentó el **RMSE**, indicando errores grandes en algunos sismos de mayor significancia.
- Aunque se usó **MSE** para penalizar errores grandes, el dataset tiene pocos eventos extremos, lo que limitó el desempeño del modelo.

Modelo 4.3: MLP con 10 capas ocultas (todas 128 neuronas)

- Se probó una red con 10 capas ocultas para representar relaciones más complejas entre las variables sísmicas.
- La mejora fue marginal y las predicciones siguieron teniendo un rango más estrecho que los valores reales.
- Esto indica que el límite principal no fue la profundidad de la red, sino la poca cantidad de eventos extremos en el dataset.

Ejecución MODELO 4 DEEP LEARNING PERCEPCIÓN

MÚLTIPLE

Modelo 4.1: MLP con capas ocultas (64, 128, 128)

Model: "sequential_7"

Layer (type)	Output Shape	Param #
dense_53 (Dense)	(None, 64)	384
dense_54 (Dense)	(None, 128)	8,320
dense_55 (Dense)	(None, 128)	16,512
dense_56 (Dense)	(None, 1)	128

Total params: 25,345 (99.00 KB)
Trainable params: 25,345 (99.00 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
24/24 — 3s 13ms/step - loss: 0.7331
Epoch 2/10
24/24 — 0s 7ms/step - loss: 0.5073
Epoch 3/10
24/24 — 0s 7ms/step - loss: 0.4528
Epoch 4/10
24/24 — 0s 15ms/step - loss: 0.4311
Epoch 5/10
24/24 — 1s 9ms/step - loss: 0.4183
Epoch 6/10
24/24 — 0s 12ms/step - loss: 0.4058
Epoch 7/10
24/24 — 0s 8ms/step - loss: 0.3988
Epoch 8/10
24/24 — 0s 18ms/step - loss: 0.3911
Epoch 9/10
24/24 — 0s 15ms/step - loss: 0.3852
Epoch 10/10
24/24 — 0s 13ms/step - loss: 0.3792

MLP_1 (10 epochs) - Escala normalizada
RMSE 0.719
MAE 0.344
MSE 0.516

MLP_1 (10 epochs) - Escala real
RMSE 216.627
MAE 103.573
MSE 46927.312

Modelo 4.2: MLP con capas ocultas (64, 64, 128, 128, 256, 256)

Model: "sequential_8"

Layer (type)	Output Shape	Param #
dense_57 (Dense)	(None, 64)	384
dense_58 (Dense)	(None, 128)	8,320
dense_59 (Dense)	(None, 128)	16,512
dense_60 (Dense)	(None, 256)	33,024
dense_61 (Dense)	(None, 256)	65,792
dense_62 (Dense)	(None, 1)	257

Total params: 124,280 (485.50 KB)
Trainable params: 124,280 (485.50 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
24/24 — 5s 11ms/step - loss: 0.6860
Epoch 2/10
24/24 — 0s 16ms/step - loss: 0.4814
Epoch 3/10
24/24 — 0s 5ms/step - loss: 0.4643
Epoch 4/10
24/24 — 0s 5ms/step - loss: 0.4607
Epoch 5/10
24/24 — 0s 5ms/step - loss: 0.4397
Epoch 6/10
24/24 — 0s 5ms/step - loss: 0.4175
Epoch 7/10
24/24 — 0s 5ms/step - loss: 0.4037
Epoch 8/10
24/24 — 0s 6ms/step - loss: 0.3938
Epoch 9/10
24/24 — 0s 5ms/step - loss: 0.3881
Epoch 10/10
24/24 — 0s 7ms/step - loss: 0.3751

MLP_2 (10 epochs) - Escala normalizada
RMSE 0.735
MAE 0.328
MSE 0.540

MLP_2 (10 epochs) - Escala real
RMSE 221.595
MAE 98.832
MSE 49104.164

Modelo 4.3: MLP con 10 capas ocultas (todas 128 neuronas)

Model: "sequential_9"

Layer (type)	Output Shape	Param #
dense_63 (Dense)	(None, 128)	768
dense_64 (Dense)	(None, 128)	16,512
dense_65 (Dense)	(None, 128)	16,512
dense_66 (Dense)	(None, 128)	16,512
dense_67 (Dense)	(None, 128)	16,512
dense_68 (Dense)	(None, 128)	16,512
dense_69 (Dense)	(None, 128)	16,512
dense_70 (Dense)	(None, 128)	16,512
dense_71 (Dense)	(None, 128)	16,512
dense_72 (Dense)	(None, 128)	16,512
dense_73 (Dense)	(None, 1)	128

Total params: 140,960 (504.00 KB)
Trainable params: 140,960 (504.00 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
24/24 — 3s 5ms/step - loss: 0.6750
Epoch 2/10
24/24 — 0s 5ms/step - loss: 0.5478
Epoch 3/10
24/24 — 0s 5ms/step - loss: 0.4900
Epoch 4/10
24/24 — 0s 6ms/step - loss: 0.4850
Epoch 5/10
24/24 — 0s 5ms/step - loss: 0.4050
Epoch 6/10
24/24 — 0s 5ms/step - loss: 0.4503
Epoch 7/10
24/24 — 0s 5ms/step - loss: 0.4301
Epoch 8/10
24/24 — 0s 6ms/step - loss: 0.4355
Epoch 9/10
24/24 — 0s 5ms/step - loss: 0.4322
Epoch 10/10
24/24 — 0s 5ms/step - loss: 0.4232

MLP_3 (10 epochs) - Escala normalizada
RMSE 0.713
MAE 0.321
MSE 0.508

MLP_3 (10 epochs) - Escala real
RMSE 214.938
MAE 96.825
MSE 46198.547

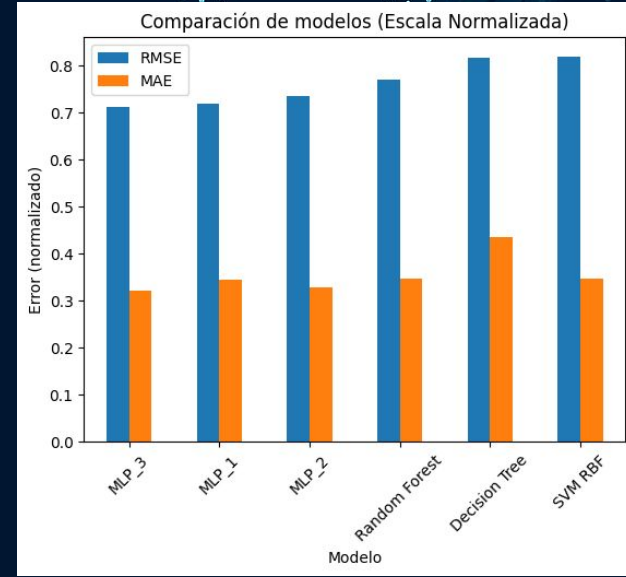
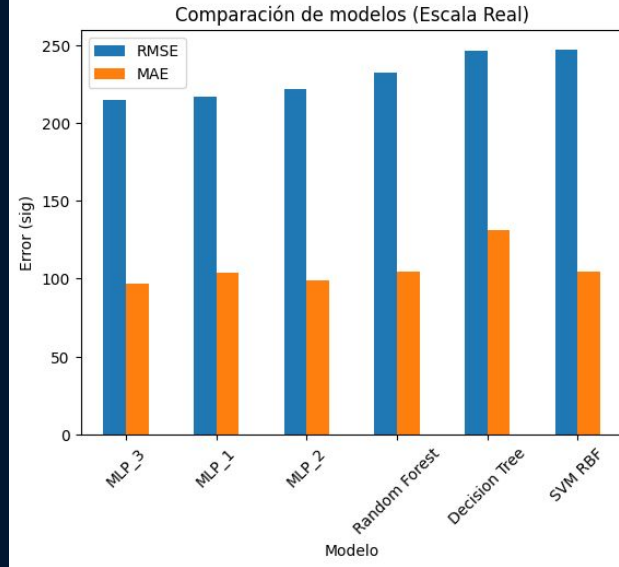
RESUMEN DE RESULTADOS: Visualización Gráfica de Modelos (Real vs Predicción)

=== RESULTADOS EN ESCALA REAL ===

	Modelo	MSE	RMSE	MAE
5	MLP_3	46198.548875	214.938472	96.825279
3	MLP_1	46927.312500	216.627128	103.573006
4	MLP_2	49104.164062	221.594594	98.832115
1	Random Forest	53920.451956	232.207778	104.602908
0	Decision Tree	60696.156478	246.365900	130.957758
2	SVM RBF	61060.383159	247.103993	104.491410

=== RESULTADOS EN ESCALA NORMALIZADA ===

	Modelo	MSE	RMSE	MAE
5	MLP_3	0.508359	0.712993	0.321188
3	MLP_1	0.516378	0.718594	0.343572
4	MLP_2	0.540331	0.735072	0.327845
1	Random Forest	0.593329	0.770278	0.347287
0	Decision Tree	0.667887	0.817244	0.434412
2	SVM RBF	0.671895	0.819692	0.346618



Escalado de datos con Min Max Scaler

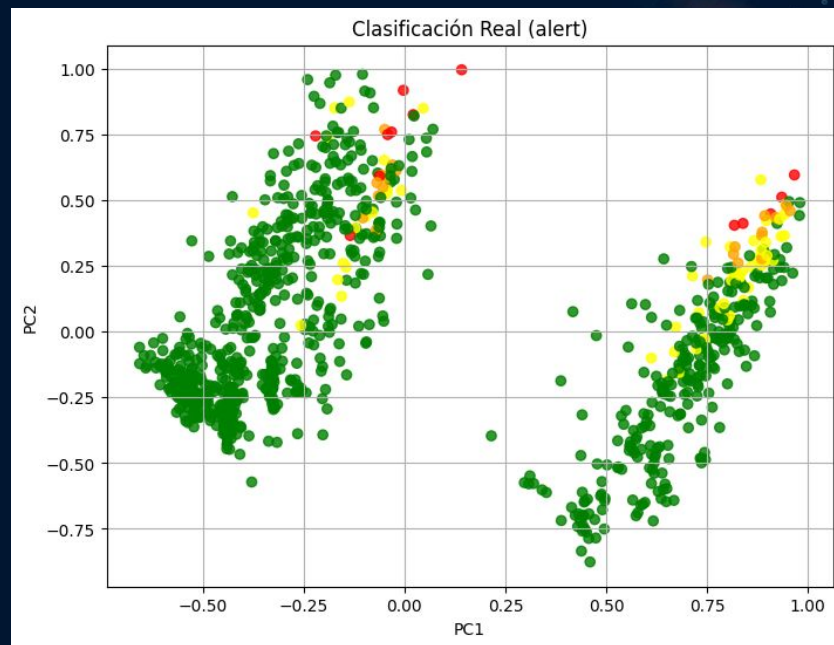
Se aplicó **MinMaxScaler** para normalizar las 11 variables numéricas en un rango entre **0** y **1**. Esto fue necesario porque algoritmos como **K-Means** y **DBSCAN** trabajan con distancias, por lo que variables con escalas mayores podían afectar el agrupamiento.

Reducción dimensional con PCA

Se aplicó **PCA** para reducir los datos a 2 componentes y visualizarlos en 2D. La variable '**alert**' se usó solo como referencia para la actividad; la gráfica muestra que los niveles de alerta no se separan claramente.

```
Varianza explicada por componente:  
[0.37774532 0.18927018]  
Varianza acumulada:  
0.5670155024623866
```

```
Shape: (1000, 11)  
Shape escalado: (1000, 11)
```



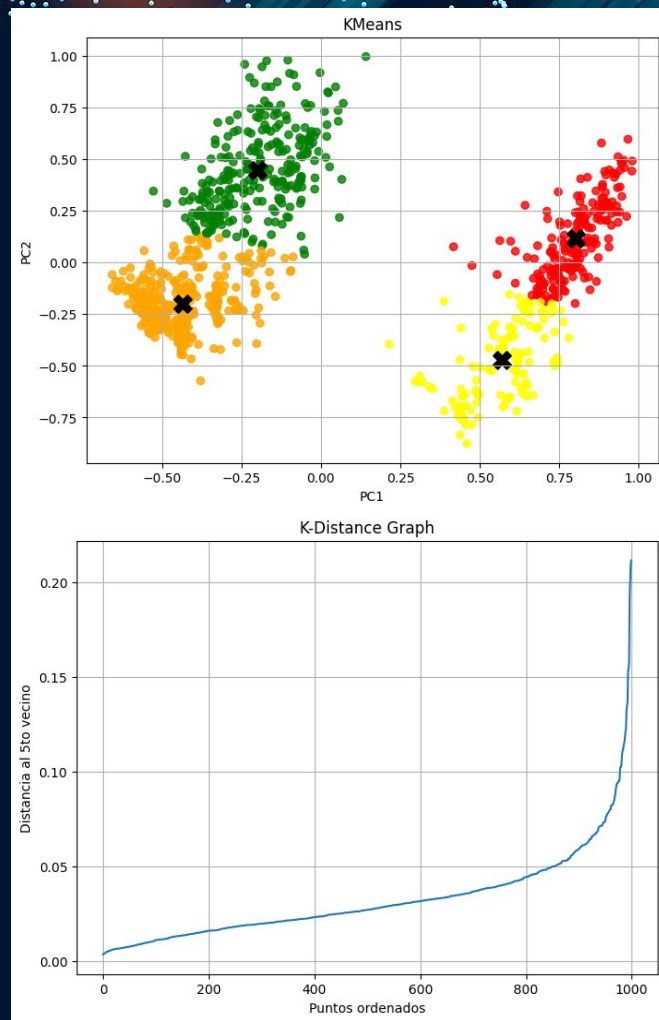
Aplicación de K-Means Clustering

Se aplicó **K-Means** sobre los datos reducidos con **PCA**, usando 4 clusters, tomando como referencia los cuatro niveles de **alert**: *green*, *yellow*, *orange* y *red*. El algoritmo agrupó los sismos según su cercanía en el espacio **PCA**, permitiendo observar si los grupos encontrados tienen alguna relación visual con los niveles de alerta.

```
Clusters KMeans:  
[0 1 2 3]
```

Selección de parámetros para DBSCAN

Antes de aplicar **DBSCAN**, se analizó el gráfico **K-Distance** para estimar un valor adecuado de **eps**. Este parámetro define la distancia máxima para formar grupos y separar posibles puntos de ruido o anomalías.



Mejor eps y minpoints

- Se probaron diferentes combinaciones de *eps* y *min_samples* para **DBSCAN**.
- La selección se hizo considerando:
 - Silhouette Score
 - Número de clusters encontrados
 - Cantidad de puntos clasificados como ruido
- La mejor configuración obtenida fue:
 - *eps* = 0.1
 - *min_samples* = 9
- Con estos valores, **DBSCAN** encontró 2 clusters y detectó 13 puntos como ruido.
- A diferencia de K-Means, **DBSCAN** no obliga a formar una cantidad fija de grupos, sino que identifica zonas densas y separa registros que no encajan claramente.

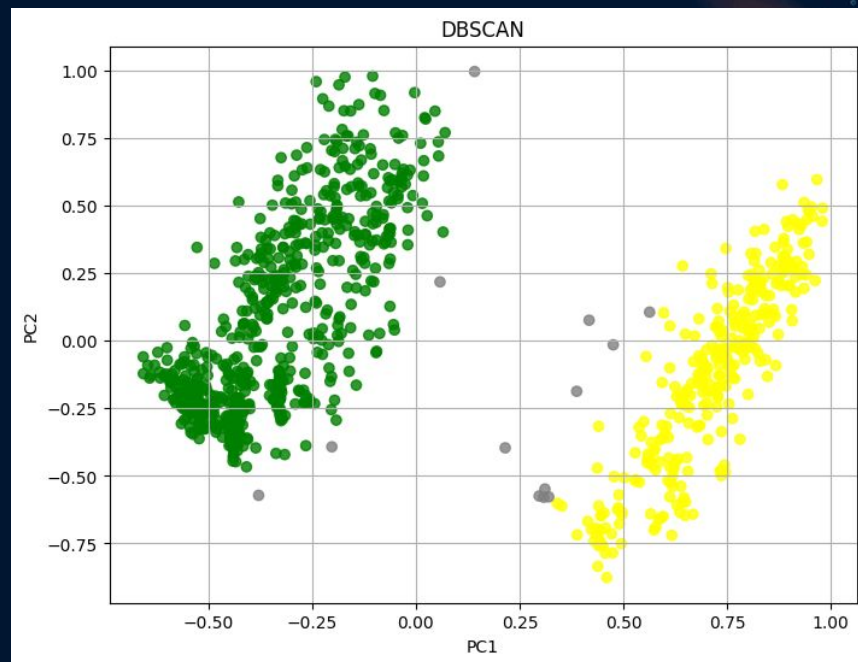
Mejor eps: 0.1

Mejor min_samples: 9

Mejor Silhouette: 0.6165943996968654

Clusters encontrados: 2

Ruido detectado: 13



Comparación para la variable alert

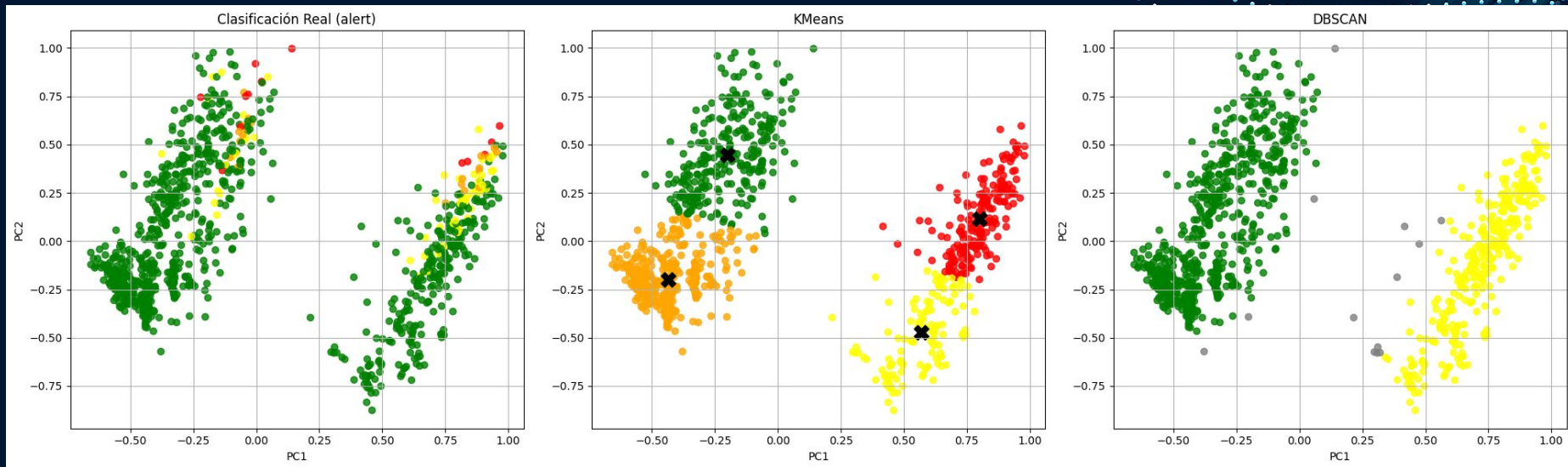
- Se compararon **K-Means** y **DBSCAN** mediante el *Silhouette Score*.
- **K-Means** obtuvo un valor de **0.604**.
- **DBSCAN** obtuvo un valor de **0.617**, considerando solo los puntos no clasificados como ruido.
- La diferencia entre ambos resultados fue pequeña.
- **DBSCAN** mostró una separación ligeramente mejor entre los grupos encontrados.

Silhouette KMeans: 0.6038786547814541

Silhouette DBSCAN: 0.6165943996968654

- **K-Means** fue configurado para formar 4 clusters, tomando como referencia los niveles de alert.
- **DBSCAN** encontró automáticamente 2 clusters y algunos puntos de ruido.
- Los niveles de **alert** no necesariamente representan agrupamientos naturales en los datos.

Comparación para la variable alert (Graficas)



Conclusiones

1. En los modelos supervisados, **Random Forest** y **SVM** con kernel *RBF* obtuvieron los mejores resultados para predecir **sig**, superando al modelo de Deep Learning en este dataset.
2. En *clustering*, **K-Means** permitió formar 4 grupos usando como referencia la variable **alert**, mientras que **DBSCAN** encontró 2 grupos naturales y detectó 13 puntos como ruido.
3. La variable **alert** se utilizó sólo como referencia para cumplir con la actividad, ya que no necesariamente representa una separación natural clara dentro de los datos.
4. En general, el proyecto comparó enfoques supervisados y no supervisados en datos sísmicos, mostrando que el análisis del impacto sísmico requiere más datos y evaluación.

